EXAMEN TP

Déploiement d’une application Full Stack sur Kubernetes — Machines virtuelles

Nom et prénom : ..... Groupe : .....

Objectif de l’examen

Construire, publier et déployer une application Angular / Spring Boot sur un cluster Kubernetes hébergé sur des machines virtuelles (VM). Externaliser la configuration avec des ressources ConfigMap et Secret, puis valider le fonctionnement de l’application.

Consignes générales

- Le seul scénario demandé est le déploiement sur Kubernetes. Aucun service AWS EC2 ni Docker Compose n’est requis.
- Utilisez des VM Ubuntu (VirtualBox, VMware ou une plateforme de virtualisation fournie par l’établissement). Le cluster Kubernetes peut être celui configuré pendant les séances de TP.
- Le code source et les Dockerfiles incomplets sont fournis. Utilisez les adresses IP privées/réseau des VM et les ports correspondant à votre environnement.
- Ne transmettez aucun identifiant, mot de passe ou jeton réel dans vos livrables.

Partie 1 — Préparation des machines virtuelles et du cluster

1.1. Infrastructure virtuelle

- Préparez les VM Ubuntu nécessaires : au minimum un nœud control-plane et un nœud worker, ou le cluster de TP existant si celui-ci est déjà opérationnel.
- Configurez leur réseau (interfaces, adresses IP, résolution de noms si nécessaire) et vérifiez la communication entre les VM. Documentez les adresses IP et le mode réseau (pont ou réseau adapté aux VM).
- Vérifiez la présence des outils requis : Git, Docker pour la construction des images, Maven si nécessaire et kubectl sur la VM d’administration. Un moteur de conteneurs compatible doit être opérationnel sur les nœuds Kubernetes.

1.2. Vérifications

```
ip -br a
ping <IP_AUTRE_VM>
docker --version
kubectl version --client
kubectl get nodes -o wide
```

- Récupérez les sources avec : git clone <URL_DU_DEPOT>. Fournissez la sortie des commandes attestant que les nœuds sont Ready.

Partie 2 — Construction et publication des images Docker

2.1. Front-end Angular : compléter le Dockerfile

Complétez les instructions manquantes en conservant une construction multi-stage (Node.js, puis Nginx).

```
FROM _____ AS build
WORKDIR _____
COPY _____
RUN _____
COPY _____
RUN npm run build --force
```

```
FROM _____
COPY nginx.conf /etc/nginx/nginx.conf
COPY --from=build /app/dist/angular-app /usr/share/nginx/html
```

Images imposées par le sujet initial : node:16-alpine et nginx:1.17.1-alpine. Vérifiez le chemin réel du répertoire de compilation Angular.

2.2. Back-end Spring Boot : compléter le Dockerfile

```
_____ _____
VOLUME /tmp

_____
ARG JAR_FILE=target/demo-0.0.1.jar
ADD ${JAR_FILE} app.jar
ENTRYPOINT ["java","-jar","/app.jar"]
```

Image imposée : OpenJDK 11. Port du service : 8081, à harmoniser avec la configuration Spring Boot.

2.3. Build et dépôt d'images

- Compilez Spring Boot avec Maven ; construisez les deux images sur la VM de travail ; identifiez-les avec les tags de votre dépôt Docker Hub et publiez-les.
- Fournissez les commandes de compilation, docker build, docker tag et docker push, et les preuves de publication.

Partie 3 — Déploiement Kubernetes sur les VM

3.1. Manifests

- Créez angular-deployment.yaml et spring-deployment.yaml, comportant les objets Deployment et Service nécessaires.
- Spécifiez les images, réplicas, labels/selectors, ports des conteneurs, variables d'environnement et stratégies d'accès aux services.
- Utilisez des Services adaptés à un cluster hébergé sur des VM : ClusterIP pour les échanges internes ; NodePort ou Ingress si configuré pour l'accès externe. Ne supposez pas qu'une adresse LoadBalancer publique est disponible.

3.2. Exécution et contrôle

```
kubectl apply -f angular-deployment.yaml
kubectl apply -f spring-deployment.yaml
kubectl get nodes,pods,deployments,services -o wide
kubectl describe pod <NOM_DU_POD>
kubectl logs <NOM_DU_POD>
```

- Ouvrez l'application à partir d'une adresse de VM accessible depuis le poste client et du NodePort (ou du nom DNS de l'Ingress). Testez l'API et les appels front-end vers le back-end.
- En cas d'échec, vérifiez le pare-feu des VM, le routage, les ports exposés, la résolution des services Kubernetes et les logs des Pods.

3.3. Fichiers applicatifs à inspecter

- Spring Boot : application.properties et configuration CORS. Angular : src/environments/ et src/app/services/. Relevez les URL/IP, ports, identifiants et paramètres de base de données codés en dur.

Partie 3 bis — Captures de référence : adresses IP à adapter

Les captures suivantes proviennent du sujet initial. Elles illustrent les emplacements des anciennes adresses 54.175.125.22. Cette IP publique est un exemple historique : elle ne correspond pas nécessairement à vos VM. Remplacez-la selon les IP de votre cluster, les noms DNS et le mode d'accès choisi.

Renseignements à fournir : IP de la VM control-plane : ..... ; IP de la VM worker : ..... ; IP du poste client : ..... ; NodePort du front-end : .....

Capture 1 — Spring Boot : CORS (CorsConfig.java)

```java
package com.oauth.demo.config;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.CorsRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class CorsConfig implements WebMvcConfigurer {

    @Override
    public void addCorsMappings(CorsRegistry registry) {
        registry.addMapping("/**")
                .allowedOrigins("http://54.175.125.22:4200")
                .allowedMethods("GET", "POST", "PUT", "DELETE", "OPTIONS")
                .allowedHeaders("*");
    }
}
```

Dans allowedOrigins, renseignez l'origine exacte du front-end accessible depuis le navigateur (schéma, hôte et port). Cette origine doit correspondre à l'URL réelle de consultation.

Capture 2 — Spring Boot : application.properties

```
server.port=8081
spring.datasource.url=jdbc:mysql://54.175.125.22:3306/demo
spring.datasource.username=root
spring.datasource.password=root
spring.jpa.hibernate.ddl-auto=update
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver


spring.mvc.cors.mapping=/**
spring.mvc.cors.allowed-origins=http://54.175.125.22:4200
spring.mvc.cors.allowed-methods=GET,POST,PUT,DELETE
spring.mvc.cors.allowed-headers=*
```

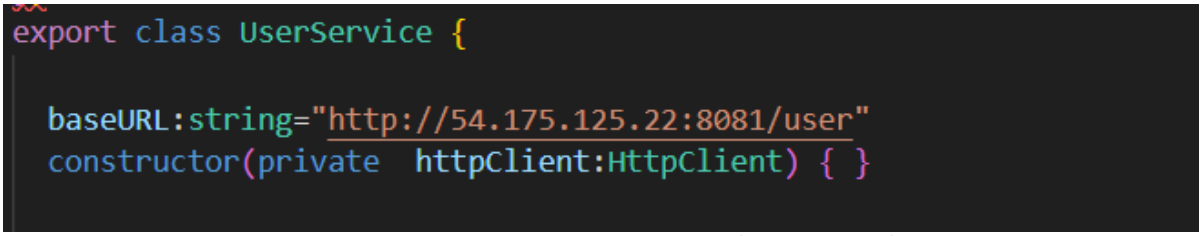
Adaptez le port de l'application et l'URL JDBC vers la base accessible depuis les VM ou le cluster. Externalisez les identifiants et mots de passe dans un Secret. Ne réutilisez pas les valeurs de démonstration affichées.

Capture 3 — Angular : src/environments/environment.ts

```typescript
export const environment = {
  production: false,
  apiUrl: 'http://54.175.125.22:8081'
};
```

Corrigez l'URL de l'API pour qu'elle soit accessible depuis le navigateur. Un nom de Service ClusterIP n'est pas directement résolvable par un navigateur extérieur au cluster.

Capture 4 — Angular : src/app/services/user.service.ts



Éliminez l'adresse IP inscrite directement dans le service Angular : réutilisez le mécanisme de configuration prévu pour l'URL de l'API.

Travail demandé : fournir les captures des fichiers corrigés, le plan d'adressage IP de vos VM, les commandes de vérification réseau et les preuves de fonctionnement des routes API. Une IP privée de VM et un NodePort ne sont utilisables depuis le poste client que si le réseau le permet.

Partie 4 — Externalisation de la configuration : ConfigMap et Secret

4.1. Identifier les variables

- Classez les paramètres : non sensibles (URL de base de données, ports, origine CORS, etc.) et sensibles (utilisateur, mot de passe, jetons).

4.2. Mettre en œuvre les ressources Kubernetes

- Créez configmap.yaml pour les données non sensibles et secret.yaml pour les données sensibles, en utilisant uniquement des valeurs fictives dans les fichiers remis.
- Adaptez spring-deployment.yaml avec env, configMapKeyRef, secretKeyRef ou envFrom pour consommer ces ressources.
- Modifiez application.properties pour lire les variables d'environnement, notamment SPRING_DATASOURCE_URL, SPRING_DATASOURCE_USERNAME et SPRING_DATASOURCE_PASSWORD.

4.3. Cas du front-end Angular

- Supprimez l'URL d'API codée en dur. Mettez en place et expliquez une configuration à la compilation, ou une véritable configuration à l'exécution (par exemple config.js servi par Nginx).

Attention : une variable d'environnement injectée dans un conteneur Nginx n'est pas automatiquement visible dans le navigateur.

4.4. Valider la nouvelle configuration

```
kubectl apply -f configmap.yaml
kubectl apply -f secret.yaml
kubectl apply -f spring-deployment.yaml
kubectl get configmaps,secrets,deployments,pods,services
```

- Vérifiez que le front-end, le back-end et la communication API fonctionnent après externalisation, sans afficher les valeurs sensibles.

Partie 5 — Questions de compréhension

- Pourquoi utilise-t-on Node.js puis Nginx pour Angular ? Expliquez WORKDIR et les deux instructions COPY.
- Quel est le rôle de EXPOSE, ENTRYPOINT et ARG JAR_FILE dans le Dockerfile Spring Boot ?
- Quelles sont les sections principales d'un Deployment ? Que fait kubectl apply -f ?
- Pourquoi choisir ClusterIP ou NodePort dans un environnement de machines virtuelles ? Quelle différence avec LoadBalancer ?
- Comment identifier un problème de connectivité entre VM, Service et Pod ?
- Quelle différence entre ConfigMap et Secret ? Pourquoi un Secret doit-il également être protégé par les autorisations Kubernetes et, selon le cluster, par le chiffrement au repos ?

Livrables attendus

- Dockerfiles finalisés, commandes de build et de publication, et images accessibles depuis le cluster.
- Manifests Kubernetes : angular-deployment.yaml, spring-deployment.yaml, configmap.yaml et secret.yaml (fictif).
- Fichiers applicatifs adaptés pour Spring Boot, CORS et Angular.
- Schéma des VM, adresses IP privées, NodePort(s), captures d’écran des quatre configurations corrigées et des tests de connectivité ; sorties de kubectl et preuves fonctionnelles.
- Réponses aux questions de compréhension.